SMART CONTRACT AUDIT

ZOKYO.

October 1st, 2021 | v. 1.0

PASS

Zokyo Security team has concluded that the given smart contracts passed security audit and are fully production-ready

TECHNICAL SUMMARY

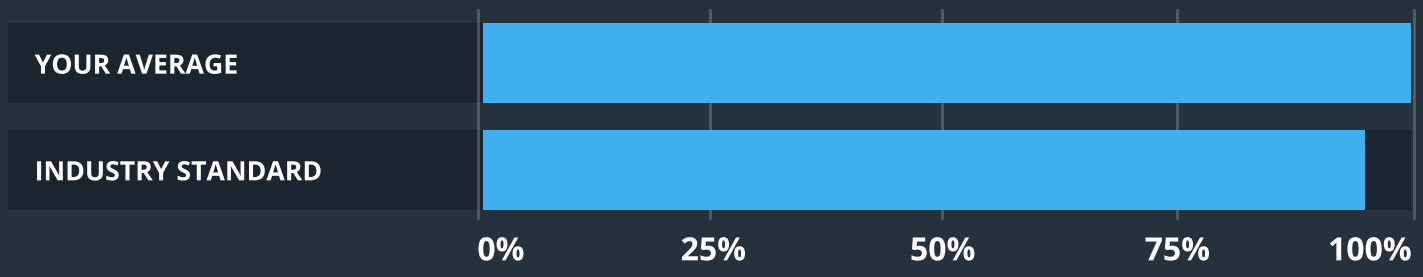
This document outlines the overall security of the Warena smart contracts, evaluated by Zokyo's Blockchain Security team.

The scope of this audit was to analyze and document the Warena smart contract codebase for quality, security, and correctness.

Contract Status

There were no critical issues found during the audit.

Testable Code



The testable code is 100%, which is above the industry standard of 95%.

It should be noted that this audit is not an endorsement of the reliability or effectiveness of the contract, rather limited to an assessment of the logic and implementation. In order to ensure a secure contract that's able to withstand the Ethereum network's fast-paced and rapidly changing environment, we at Zokyo recommend that the Warena team put in place a bug bounty program to encourage further and active analysis of the smart contract.

TABLE OF CONTENTS

Auditing Strategy and Techniques Applied . . . 3
Summary . . . 5
Structure and Organization of Document . . . 6
Complete Analysis . . . 7
Code Coverage and Test Results for all files . . . 9
Tests written by Zokyo Secured team . . . 9

AUDITING STRATEGY AND TECHNIQUES APPLIED

The Smart contract's source code was taken from the Warena repository.

Repository:
https://github.com/warenaofficial/warena-contracts/blob/master/contracts/Rena.sol

Last commit:
610b5d28d3ac4ccd58de8fa9abdace98e375dfa2

A contract under the scope:
Rena.sol

Throughout the review process, care was taken to ensure that the token contract:

- Implements and adheres to existing Token standards appropriately and effectively;
- Documentation and code comments match logic and behavior;
- Distributes tokens in a manner that matches calculations;
- Follows best practices in efficient use of gas, without unnecessary waste;
- Uses methods safe from reentrance attacks;
- Is not affected by the latest vulnerabilities;
- Whether the code meets best practices in code readability, etc.

Zokyo's Security Team has followed best practices and industry-standard techniques to verify the implementation of smart contracts. To do so, the code is reviewed line-by-line by our smart contract developers, documenting any issues as they are discovered. Part of this work includes writing a unit test suite using the Truffle testing framework. In summary, our strategies consist largely of manual collaboration between multiple team members at each stage of the review:

1	Due diligence in assessing the overall code quality of the codebase.
2	Cross-comparison with other, similar smart contracts by industry leaders.
3	Testing contract logic against common and uncommon attack vectors.
4	Thorough, manual review of the codebase, line-by-line.

SUMMARY

There were no critical, high, low, and medium issues found during the audit. Zokyo auditing team has found one informational issue, which was successfully resolved by the Warena team. The issue is described in the “Complete Analysis” section in this document.

Contracts are well written and structured. We can state that the findings during the audit have no impact on contract performance or security. Based on the audit conducted and the quality of the codebase, we can give the score of 100 to contract under the scope.

Zokyo security team assures that the given contract is fully production-ready.

STRUCTURE AND ORGANIZATION OF DOCUMENT

For ease of navigation, sections are arranged from most critical to least critical. Issues are tagged “Resolved” or “Unresolved” depending on whether they have been fixed or addressed. Furthermore, the severity of each issue is written as assessed by the risk of exploitation or other unexpected or otherwise unsafe behavior:

Critical

The issue affects the ability of the contract to compile or operate in a significant way.

High

The issue affects the ability of the contract to compile or operate in a significant way.

Medium

The issue affects the ability of the contract to operate in a way that doesn’t significantly hinder its behavior.

Low

The issue has minimal impact on the contract’s ability to operate.

Informational

The issue has no impact on the contract’s ability to operate.

COMPLETE ANALYSIS

Useless import and inheritance of ERC20.sol

INFORMATIONAL | RESOLVED

ERC20Burnable contract has already inherited ERC20.sol so there isn't any need to inherit ERC20Burnable with ERC20.

Recommendation:
Remove the inheritance of ERC20 and import of ERC20.

	Rena.sol
Re-entrancy	Pass
Access Management Hierarchy	Pass
Arithmetic Over/Under Flows	Pass
Unexpected Ether	Pass
Delegatecall	Pass
Default Public Visibility	Pass
Hidden Malicious Code	Pass
Entropy Illusion (Lack of Randomness)	Pass
External Contract Referencing	Pass
Short Address/ Parameter Attack	Pass
Unchecked CALL Return Values	Pass
Race Conditions / Front Running	Pass
General Denial Of Service (DOS)	Pass
Uninitialized Storage Pointers	Pass
Floating Points and Precision	Pass
Tx.Origin Authentication	Pass
Signatures Replay	Pass
Pool Asset Security (backdoors in the underlying ERC-20)	Pass

CODE COVERAGE AND TEST RESULTS FOR ALL FILES

Tests written by Zokyo Security team

As part of our work assisting Warena in verifying the correctness of their contract code, our team was responsible for writing integration tests using the Truffle testing framework.

Tests were based on the functionality of the code, as well as a review of the Warena contract requirements for details about issuance amounts and how the system handles these.

Code Coverage

The resulting code coverage (i.e., the ratio of tests-to-code) is as follows:

FILE	% STMTS	% BRANCH	% FUNCS	% LINES	UNCOVERED LINES
contracts\	100.00	100.00	100.00	100.00	
Rena.sol	100.00	100.00	100.00	100.00	
All files	**100.00**	**100.00**	**100.00**	**100.00**	

Test Results

```
Contract: WarenaToken test
  Check for correct deployment
    ✓ Check properties of token (608ms)
    ✓ Check balance on addresses (273ms)
  Functions
    transfer
      ✓ Must fail if try to transfer to zero address (2441ms)
      ✓ Must fail if try to transfer more than sender has (504ms)
      ✓ Must transfer correctly (1861ms)
    approve and transferFrom
      ✓ Must fail if sender isn't approved (825ms)
      ✓ Must fail if sender exceeds allowance (671ms)
```

```
    ✓ Must tranferFrom correctly (977ms)
  increaseAllowance and decreaseAllowance
    ✓ Must fail if increase allowance more than max value of uint256 (overflow) (601ms)
    ✓ Must fail if decrease allowance more than was approved (underflow) (733ms)
    ✓ Must increase allowance correctly (650ms)
    ✓ Must decrease allowance correctly (646ms)
  burn
    ✓ Must fail if burn tokens more than user has (445ms)
    ✓ Must burn tokens correctly (437ms)
  burnFrom
    ✓ Must fail if sender try to burn more tokens than allowed (212ms)
    ✓ Must fail if sender try to burn tokens from zero address (221ms)
    ✓ Must fail if sender try to burn tokens from zero address (210ms)
    ✓ Must fail if sender try to burn more tokens than the address has (647ms)
    ✓ Must burnFrom correctly (762ms)

19 passing (17s)
```

We are grateful to have been given the opportunity to work with the Warena team.

The statements made in this document should not be interpreted as investment or legal advice, nor should its authors be held accountable for decisions made based on them.

Zokyo's Security Team recommends that the Warena team put in place a bug bounty program to encourage further analysis of the smart contract by third parties.

ZOKYO.